Workspace Indexing: Remote vs. Local Semantic Search

Question: A developer opens a large repository with 12,000 files in VS Code and asks a question using `@workspace`. How does GitHub Copilot determine which code snippets to retrieve when the repository is hosted on GitHub.com and has a pre-built remote code search index?

A. Copilot uploads all 12,000 files inside a single LLM prompt window on every chat turn.

B. Copilot queries the remote GitHub semantic code index to retrieve the most relevant chunks, supplemented by uncommitted local edits. **[CORRECT ANSWER]**

C. Copilot ignores the remote GitHub repository and only searches currently open editor tabs.

D. Copilot requires manual creation of a `.copilot-index.json` manifest before `@workspace` can run.

Explanation & Exam Rationale

Because LLM context windows have strict token limits, `@workspace` relies on an indexing and retrieval pipeline rather than sending an entire repository. When a workspace maps to a GitHub-hosted repository with a remote index enabled, Copilot queries that remote semantic index for fast, high-accuracy retrieval across large codebases and merges those results with local uncommitted changes.

Index Type	When Used	Characteristics
Remote Index	Repository is hosted on GitHub.com & indexed	Fast server-side semantic search; scales to very large repos; combines with local diff.
Local Index	Non-GitHub repo or remote index unavailable	Built locally on the developer's machine (typically supports smaller/medium workspaces).
Basic Lexical Fallback	Very large unindexed local workspace	Falls back to lexical/keyword matching and open file symbols when local indexing limits are exceeded.

Key Exam Takeaway: Understanding the hierarchy of **Remote Index → Local Index → Lexical Fallback** is critical for Domain 04 questions on how `@workspace` scales to enterprise repositories.